Computational Complexity of the Unit Commitment Problem

Muhy Eddin Zater

May 11, 2026

Abstract

This note develops a layered computational complexity analysis of the Unit Commitment (UC) problem in power systems. UC is a Mixed-Integer Linear Program (MILP) whose worst-case complexity is exponential in the product $N_g \times T$ of generator count and horizon length. We derive a master cost expression that decomposes branch-and-cut UC solves into three multiplicative levels—tree exploration, per-node LP work, and the linear-algebra interior of the LP solver—and characterize how formulation choice, decomposition methods, and machine learning warm-starts modify each level. We then map asymptotic estimates against observed wall-clock runtimes on canonical benchmarks (small test networks through ERCOT-scale day-ahead SCUC), and analyze the CPU-vs-GPU trade-off through the lens of memory access patterns and recent first-order methods (PDHG).

1 Introduction

The Unit Commitment problem schedules the on/off status, startup, shutdown, and power output of a fleet of generators over a discrete horizon to satisfy demand and reserve requirements at minimum cost, subject to per-unit physical limits (minimum up/down times, ramp rates, capacity bounds) and system-wide constraints. Day-ahead and intraday operation in essentially every modern electricity market is built around solving UC instances. The *security-constrained* variant (SCUC) further requires the schedule to remain feasible under a list of contingency scenarios, multiplying the constraint count by orders of magnitude.

UC is hard for two related reasons. The integer commitment variables make the problem combinatorial—a fact that places it formally in the NP-hard class. System-wide coupling constraints (demand balance, reserve, and transmission) prevent decomposition into independent unit-level subproblems. Practical solution depends on the interaction of four design choices, which this document treats as separate layers:

1. The problem class, which is fixed by the mathematical structure;
2. The formulation, which selects a particular MILP encoding of the same problem;
3. The relaxation and decomposition strategy, which produces tractable auxiliary problems whose solutions guide the search;
4. The solver and hardware, which determine the wall-clock realization of the algorithm.

The first three items shape the asymptotic complexity; the fourth determines the effective constants. Section 2 fixes notation and the canonical formulation. Section 3 states the master cost expression. Sections 4–6 analyze each level. Sections 7–8 examine formulation tightness and decomposition methods. The remainder addresses theory-vs-runtime, hardware, and case studies.

2 Problem Statement and Formulation

2.1 Decision variables

For each generator $i \in \{1, \dots, N_g\}$ and time period $t \in \{1, \dots, T\}$:

- $u_{it} \in \{0, 1\}$: commitment status (on/off),
- $v_{it} \in \{0, 1\}$: startup indicator,
- $w_{it} \in \{0, 1\}$: shutdown indicator,
- $p_{it} \in \mathbb{R}_{\geq 0}$: power output above minimum.

2.2 Three-bin formulation

The canonical (three-bin) formulation is

$$\min \sum_{i,t} \left[c_i^{\mathrm{nl}} u_{it} + c_i^{\mathrm{var}}(p_{it}) + c_i^{\mathrm{su}} v_{it} + c_i^{\mathrm{sd}} w_{it} \right] \tag{1}$$

subject to a state equation linking the binaries,

$$u_{it} - u_{i,t-1} = v_{it} - w_{it}, \qquad v_{it} + w_{it} \leq 1, \tag{2}$$

generator capacity, ramping, and minimum up/down-time constraints,

$$\underline{P}_i u_{it} \leq P_{it} \leq \overline{P}_i u_{it}, \qquad P_{it} - P_{i,t-1} \leq R_i^{\uparrow} u_{i,t-1} + R_i^{\mathrm{su}} v_{it}, \tag{3}$$

$$\sum_{\tau=t-\mathrm{UT}_i+1}^{t} v_{i\tau} \leq u_{it}, \qquad \sum_{\tau=t-\mathrm{DT}_i+1}^{t} w_{i\tau} \leq 1 - u_{it}, \tag{4}$$

and system-wide coupling,

$$\sum_i P_{it} = D_t, \qquad \sum_i \overline{P}_i u_{it} - \sum_i P_{it} \geq R_t^{\mathrm{res}}. \tag{5}$$

The *coupling constraints* (5)—demand balance and reserve adequacy, plus transmission limits in network-aware variants—are the source of combinatorial difficulty: they prevent the per-generator subproblems from decoupling.

2.3 Counting the discrete state space

The total number of (u_{it}) patterns is $2^{N_g T}$. With ramp and minimum-up/down constraints, the feasible subset is much smaller, but its *size* still grows exponentially in $N_g T$. This places UC in the NP-hard class: no algorithm is known that solves arbitrary instances in polynomial time, and one is widely conjectured not to exist.

2.4 Notation

Symbol	Meaning
N_g	number of generators
T	horizon length (time periods)
$n = N_g T$	total binary variable count
N_{nodes}	branch-and-bound nodes explored
$\bar{C}_{\text{node}}$	average per-node cost (LP reoptimization + bookkeeping)
g_{LP}	LP relaxation gap at root, $(z^* - z^{\text{LP}})/z^*$
d_{tree}	effective tree depth (number of fractional binaries)
$\text{nnz}(\cdot)$	number of nonzeros

Table 1: Notation used throughout.

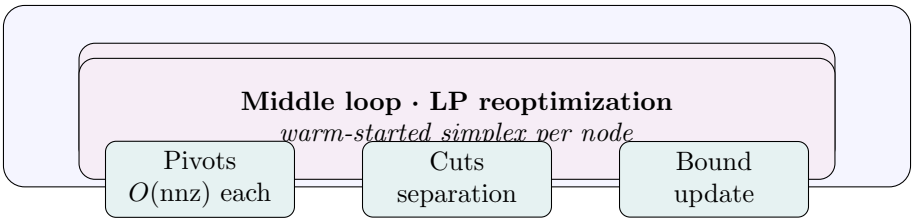
3 Master Cost Expression

A modern MILP solver (Gurobi, CPLEX, HiGHS, SCIP) attacks UC with branch-and-cut. Three multiplicative levels stack inside one another. The total cost is

$$C_{\text{total}} = C_{\text{presolve}} + C_{\text{root}} + \underbrace{N_{\text{nodes}}}_{\text{tree size}} \times \underbrace{\bar{C}_{\text{node}}}_{\text{per-node LP work}} + C_{\text{cuts}} + C_{\text{heur}} \tag{6}$$

where the dominant term is almost always $N_{\text{nodes}} \cdot \bar{C}_{\text{node}}$ on instances where UC is genuinely difficult, and the root LP plus presolve dominates on instances where it is easy. The per-node cost itself decomposes into LP solver work, which we analyze separately.

The structure looks like this:



4 Level 1: Search Tree Size

The outer-loop multiplier N_{nodes} has worst-case bound 2^n but a practical value driven by the strength of the relaxation and the quality of heuristics. Understanding what controls N_{nodes} is the single most important question in UC performance.

4.1 The branch-and-bound mechanism

At each tree node, the solver:

1. Solves the LP relaxation (binaries replaced by $0 \le u_{it} \le 1$), producing a lower bound $z^{\text{LP}}_{\text{node}}$.

2. If $z^{\text{LP}}_{\text{node}} \geq z^{\text{incumbent}}$, the node is *pruned*: no integer solution beneath this node can improve the incumbent.

3. Otherwise, if the LP solution is fractional, the solver *branches* on a fractional variable, creating two child nodes with $u_{it} = 0$ and $u_{it} = 1$. If the LP solution is integer, it becomes a candidate incumbent.

4.2 What controls tree size

Tree size is governed by the interaction of three quantities:

- **Root LP gap** $g_{\text{LP}} = (z^* - z^{\text{LP}})/z^*$. A loose relaxation places the root bound far below the optimum, so many candidate nodes appear competitive and few are pruned. A formulation that tightens g_{LP} from 5% to 0.5% typically reduces N_{nodes} by one to three orders of magnitude.
- **Incumbent quality**. Pruning is more aggressive when the incumbent is good, because more lower bounds clear the threshold. Heuristics (feasibility pump, RINS, local branching, ML-based warm starts) supply incumbents early.
- **Branching strategy**. Strong branching, pseudo-cost branching, and reliability branching trade per-node work for tighter tree exploration. Modern solvers default to reliability branching for MILPs at UC's scale.

4.3 Worst-case versus practical scaling

Theoretical:

$$N^{\text{worst}}_{\text{nodes}} = O(2^n) = O(2^{N_g T}). \tag{7}$$

Empirical, on production-grade UC instances with a tight formulation and modern solver:

$$N^{\text{empirical}}_{\text{nodes}} = O(n^\gamma), \qquad \gamma \in [1, 3] \tag{8}$$

on instances that solve to optimality within their time budget.[1] The gap between (7) and (8) is the entire reason UC is solvable at industrial scale.

Contribution. The dominant exponential factor in C_{total}. Every other engineering choice in UC is, in the end, an attempt to reduce N_{nodes}.

5 Level 2: Per-Node Cost

Inside each branch-and-bound node, the dominant work is solving the LP relaxation. Two facts shape this cost.

[1] This empirical scaling is observed only on the subset of instances the solver *can* finish—the asymptotic behavior is dominated by instances that hit the time limit, where the worst-case bound is the only honest one.

5.1 Warm-started LP reoptimization

Child-node LPs differ from the parent by a single bound change ($u_{it} = 0$ or $u_{it} = 1$). Dual simplex from the parent's basis converges in a small number of pivots—typically 10–100, often much less than the "cold-start" simplex iteration count of $O(n)$ to $O(n^{1.5})$. Per-pivot cost is $O(\text{nnz}(A))$ where A is the constraint matrix. Net per-node cost is

$$\bar{C}_{\text{node}} \approx k_{\text{pivot}} \cdot \text{nnz}(A), \tag{9}$$

with $k_{\text{pivot}} \ll n$ for warm-started reoptimization.

5.2 Sparsity in UC matrices

UC constraint matrices are dominated by:

- *Generator block-diagonal* structure: per-unit ramp, minimum-up/down, and capacity constraints couple variables only within a single generator across time. This contributes $O(N_g T)$ nonzeros.
- *Coupling rows*: demand and reserve at each time step, plus transmission rows in network UC. These rows have $O(N_g)$ entries and there are $O(T)$ of them (or $O(T \cdot N_{\text{lines}})$ in network UC).

Total: $\text{nnz}(A) = O(N_g T)$ for non-network UC, $O(T \cdot N_{\text{lines}} \cdot d_{\text{shift}})$ added in DC-network UC where d_{shift} is the average density of the PTDF or shift-factor matrix. SCUC adds $O(N_{\text{cont}})$ copies of the network rows, blowing the matrix up by the contingency count.

5.3 Per-node cost: orders of magnitude

For a small UC ($N_g = 50$, $T = 24$): $\text{nnz}(A) \sim 10^4$, warm-started LP in $\sim 10^5$ flops, sub-millisecond per node.

For a large day-ahead UC ($N_g = 3{,}000$, $T = 24$): $\text{nnz}(A) \sim 10^6$, warm-started LP in $\sim 10^7$–10^8 flops, $\sim$ 10–100 ms per node.

For SCUC at ERCOT scale with $N_{\text{cont}} \sim 10^3$: $\text{nnz}(A) \sim 10^9$, per-node cost rises into the seconds-per-node range, which is why SCUC is not solved as a monolith but via Benders decomposition.

Contribution. Linear in N_{nodes}. The dominant question is the leading constant, controlled by warm-start quality and matrix sparsity.

6 Level 3: LP Solver Internals

The per-node LP cost decomposes further into solver-algorithmic terms. Three methods compete.

6.1 Simplex method

- Per-pivot cost: $O(\text{nnz}(A))$ for the ratio test, plus the LU update of the basis factorization, which costs $O(\text{nnz}(L + U))$ amortized.
- Pivot count: empirically scales as $O(n)$ to $O(n^{1.5})$ on non-degenerate LPs from cold start; $O(1)$ to $O(\sqrt{n})$ from warm start.

- Dual simplex is preferred at branch-and-bound nodes because bound changes leave the dual basis feasible.
- Memory access pattern: irregular, dominated by sparse pivots and LU updates—**latency-bound**, with little benefit from SIMD.

6.2 Barrier (interior-point) method

- Solves a sequence of Newton systems on a perturbed KKT system $M\,\Delta z = -r$, with $M = A^\top \Theta A$ for diagonal Θ.
- Per-iteration cost: dominated by the Cholesky factorization of M, $O(\text{nnz}(A) + (\text{fill-in}))$.
- Iteration count: $O(\log(1/\varepsilon))$ to converge to relative gap ε, typically 30–80 iterations regardless of problem size.
- Memory access: regular within the factorization, irregular within the graph traversal of M. Faster than simplex on large dense LPs; comparable on power-system LPs.
- Cannot warm-start from a parent basis, so used at the root node and occasionally for restart, not at every B&B node.

6.3 Primal-Dual Hybrid Gradient (PDHG)

A first-order method that has reshaped large-scale LP since 2021:

- Replaces matrix factorization with iterative matrix-vector products Ax and $A^\top y$.
- Per-iteration cost: $O(\text{nnz}(A))$, with the work expressed entirely as SpMV.
- Iteration count: $O(1/\varepsilon)$ to relative gap ε, with typical counts in the 10^3–10^5 range. This is far worse than barrier in iteration count but each iteration is far cheaper.
- Memory access: **bandwidth-bound**, regular streaming of A through processors. This is the regime GPUs are built for.
- Used in cuOpt (NVIDIA) and recent Gurobi/HiGHS releases as a hybrid root solver: PDHG to medium accuracy on GPU, crossover to a simplex basis on CPU, then standard branch-and-cut.

6.4 Per-node solver choice

Method	Per-iter	Iter count	Memory pattern	Warm start
Dual simplex	$O(\text{nnz})$	$O(1)$–$O(\sqrt{n})$ warm	latency-bound	yes
Primal simplex	$O(\text{nnz})$	$O(n)$–$O(n^{1.5})$ cold	latency-bound	yes
Barrier	$O(\text{nnz} + \text{fill})$	$O(\log 1/\varepsilon)$	mixed	no
PDHG	$O(\text{nnz})$	$O(1/\varepsilon)$	bandwidth-bound	partial

Table 2: LP method characteristics relevant to UC.

In a typical B&C run, dual simplex handles the vast majority of nodes; barrier or PDHG solves the root LP for tight initial bounds.

Contribution. Sets the leading constant of $\bar{C}_{\text{node}}$. Algorithmic choice here can swing per-node runtime by an order of magnitude without changing N_{nodes}.

7 Formulation Tightness

Three MILP encodings of UC—all representing the same NP-hard problem, all with 2^n worst-case nodes—differ dramatically in N_{nodes} because they differ in g_{LP}.

7.1 Three-bin formulation

The canonical encoding from Section 2.

- LP gap: typically 1–5% on standard benchmark UC instances.
- Strength of min-up/down constraints in this form is poor; LP solutions exhibit fractional commitments such as $u_{it} = 0.5$ that are far from any integer solution.
- Standard baseline; what every solver receives if no tightening is done.

7.2 Perspective formulation

Replaces the relaxed quadratic cost $C(P_{it}) = aP_{it}^2 + bP_{it} + c$ with the perspective function $u_{it} \cdot C(P_{it}/u_{it})$, equivalent to the SOCP constraint $q_{it}u_{it} \geq P_{it}^2$.

- Geometrically, this replaces the loose linear envelope of the cost curve with the convex hull of the union of the operating curve and the origin, forcing cost to zero exactly when $u_{it} = 0$.
- LP gap reduction: typically 0.3–1% from the three-bin baseline, closing 50–80% of the original optimality gap at the root node.
- Trade-off: per-node solve becomes an SOCP, $\sim$ 2–5$\times$ heavier than an LP. The reduction in N_{nodes} usually pays for it.

7.3 Set-partitioning (path-based) formulation

Each variable $\lambda_{ik} \in \{0, 1\}$ selects a complete T-period schedule (a "path") for generator i. Per-unit constraints are absorbed into the enumeration of feasible paths; the master problem retains only the partitioning constraint $\sum_k \lambda_{ik} = 1$ per generator and the system coupling.

- LP gap: near zero. The path enumeration captures the convex hull of each unit's feasible region by construction.
- Variable count: exponential—$O(2^T)$ per generator in the worst case. Not enumerated upfront; generated lazily by *column generation* (Dantzig–Wolfe decomposition), with a single-unit pricing subproblem solved by dynamic programming in $O(T \cdot S_i)$ where S_i is the per-unit state space.

- Tractability: the master LP is small once a basis of relevant paths is known, but the column-generation cost can dominate. Used mostly in research codes and a few industrial niches; mainstream solvers handle the underlying problem more directly.

7.4 Tightness summary

Formulation	LP gap	Per-node cost	Practical N_{nodes}
Three-bin (vanilla)	1–5%	LP, light	high
Three-bin + facet-defining cuts	0.3–1%	LP, medium	medium
Perspective	0.3–1%	SOCP, $\sim 3\times$ LP	medium
Set-partitioning + CG	$\approx 0\%$	LP + DP pricing	low (heavy upfront)

Table 3: Formulation strength and the tradeoff with per-node cost.

The right answer for production UC at the moment is three-bin plus facet-defining cuts, often supplemented by perspective constraints on the quadratic cost component, with branch-and-cut left to do the rest.

8 Decomposition Methods

When monolithic UC stops fitting in time or memory, decomposition splits it into more tractable pieces. Three frameworks dominate.

8.1 Lagrangian relaxation

Dualizes the system-coupling constraints (5) into the objective with multipliers λ_t:

$$\mathcal{L}(\lambda) = \min_{u,P} \sum_{i,t} c_i(\cdot) + \sum_t \lambda_t \big(D_t - \sum_i P_{it}\big). \tag{10}$$

The relaxed problem decomposes into N_g independent single-unit subproblems, each solvable by dynamic programming in $O(T \cdot S_i)$ time. The dual master maximizes $\mathcal{L}(\lambda)$ via subgradient or bundle methods, typically 50–200 iterations.

- Per-iteration cost: $\sum_i O(TS_i) \sim O(N_g T)$.
- Total: $O(k_{\text{LR}} \cdot N_g T)$ with $k_{\text{LR}} \sim 10^2$.
- Provides a strong lower bound and a near-feasible primal solution ("Lagrangian heuristic"), but a feasibility repair step is required to satisfy demand and reserves exactly.
- State-of-the-art for monolithic UC from the 1980s through the early 2000s, displaced by branch-and-cut as MILP solvers matured.

8.2 Benders decomposition

The standard approach for SCUC, where each contingency multiplies the constraint count.

- **Master problem**: a UC MILP with the original commitment variables and a placeholder for re-dispatch costs across contingencies.
- **Subproblems**: one LP per contingency (DC OPF re-dispatch), small and independent, solved in parallel.
- **Cuts**: each subproblem returns either a feasibility cut (this commitment is infeasible under contingency c) or an optimality cut (the true re-dispatch cost is at least this much). Cuts are added to the master.
- Convergence: typically 10–50 Benders iterations on production SCUC. Each iteration costs one master solve plus N_{cont} subproblem solves, the latter trivially parallelizable.

The complexity savings come from never assembling the monolithic constraint matrix: instead of N_{cont} network rows in one MILP, the solver handles one network's rows in the master and N_{cont} small LPs at the leaves.

8.3 Column generation

For set-partitioning formulations: at each iteration, solve the restricted master LP, use its dual prices to define a per-unit pricing subproblem, and add the most negative-reduced-cost path as a new column. Dual prices play the same coupling-coordination role as Lagrangian multipliers.

- Iteration cost: master LP (small) + N_g pricing subproblems (DP, each $O(TS_i)$).
- Total: $O(k_{\text{CG}} \cdot N_g T)$ with $k_{\text{CG}} \sim 10$–10^2.
- Used inside branch-and-price frameworks for niche industrial UC and in academic research; rarely the front-line tool for large markets.

9 Composite Complexity Expression

Combining all levels, the total UC solve cost is

$$C_{\text{total}} \approx \underbrace{C_{\text{root}}}_{\text{root LP / heuristics}} + \underbrace{N_{\text{nodes}}(g_{\text{LP}}, q_{\text{heur}})}_{\text{tree size}} \cdot \underbrace{\left[\, k_{\text{pivot}} \cdot \text{nnz}(A) \,\right]}_{\text{per-node LP}} + \underbrace{\sum_{\text{events}} C_{\text{cut}} + C_{\text{heur}}}_{\text{B\&C extras}} \tag{11}$$

with

- N_{nodes} growing as $O(2^n)$ in the worst case but $O(n^\gamma)$ for $\gamma \in [1, 3]$ in practice on solvable instances;
- $\text{nnz}(A) = O(N_g T)$ for monolithic UC, $O(T(N_g + N_{\text{lines}} + N_{\text{cont}} N_{\text{lines}}))$ for SCUC;
- $k_{\text{pivot}} = O(1)$–$O(\sqrt{n})$ for warm-started dual simplex.

The exponential factor is the wall: it cannot be removed, only delayed. Formulation tightening, cutting planes, and warm starts all attack N_{nodes}. Decomposition methods restructure the problem so that exponential growth is confined to a smaller subproblem.

10 Constraint Hierarchy and Problem Scale

Real-world UC formulations are built up by adding successive constraint classes to a polynomial-time core. Each layer changes the variable count, the nonzero count of the constraint matrix, and—in some cases—the complexity class of the problem itself. This section traces the progression from economic dispatch to full SCUC and beyond, showing how each layer interacts with the master cost expression (11).

10.1 The polynomial core: economic dispatch

The simplest version of the problem is *economic dispatch* (ED): given a fixed commitment u_{it}, find the cheapest power output P_{it} to meet demand. With linear or quadratic costs and capacity bounds only, ED is an LP (or QP) and admits polynomial-time solution.

- Variables: $N_g T$ continuous.
- Constraints: T demand balance + $2N_g T$ capacity bounds.
- $\text{nnz}(A) = O(N_g T)$.
- Complexity: $O(n^{1.5})$ to $O(n^3)$ via simplex or barrier.
- Wall clock at ERCOT scale ($N_g \sim 700$, $T = 24$, $n \sim 17{,}000$): seconds.

This is the polynomial floor below which UC's exponential structure does not appear. Every subsequent layer either adds variables, adds constraints, or introduces integrality.

10.2 Adding integrality: the canonical UC

Introducing the binary commitment $u_{it} \in \{0, 1\}$ and the startup/shutdown logic of equation (2) converts ED into UC. Variable count triples and constraint count roughly doubles, but the qualitative change is the jump from P to NP-hard.

- Added: $3N_g T$ binaries, $\sim 2N_g T$ state-equation rows.
- $\text{nnz}(A)$: $\sim 5N_g T$.
- Complexity: $O(2^{N_g T})$ worst case; $O(n^\gamma)$ empirically with $\gamma \in [1, 3]$ on solvable instances.
- Wall clock at ERCOT scale: 30 seconds to a few minutes.

10.3 Intertemporal coupling: ramps and minimum up/down times

These constraints couple time periods within a single generator and are the defining physical features that distinguish UC from a sequence of independent ED problems.

- **Minimum up/down times**: $\sim 2N_g T$ rows, each spanning UT_i or DT_i time steps—$O(N_g T \cdot \overline{\text{UT}})$ nonzeros total, with $\overline{\text{UT}} \sim$ 4–12 hours.
- **Ramp rate constraints**: $2N_g T$ rows, each with two nonzeros (consecutive-period coupling).

- **Startup/shutdown ramp**: $2N_gT$ rows, three nonzeros each.

The variable count is unchanged. The constraint count grows by a small multiplicative factor. The much more important effect is on g_{LP}: weak min-up/down formulations leave the LP relaxation loose, while facet-defining versions (Rajan–Takriti, Ostrowski et al.) can tighten the gap by an order of magnitude. **This is a case where adding constraints makes the problem solve faster**—the tighter LP relaxation shrinks N_{nodes} more than the per-node cost grows.

10.4 Reserves and ancillary services

Operating reserve, regulating reserve, replacement reserve, and ramp products add per-product variables and coupling rows.

- Added variables: $N_{\text{prod}}N_gT$ continuous (reserve allocation per unit per product per time).
- Added rows: $N_{\text{prod}}T$ system reserve adequacy + $N_{\text{prod}}N_gT$ unit-level allocation limits.
- nnz(A) growth: $O(N_{\text{prod}}N_gT)$.
- Practical N_{prod}: 3–8 products in modern markets.
- Wall-clock impact: 1.5–3× from baseline UC.

Reserves rarely cause qualitative scaling problems on their own; their main effect is the constant-factor blow-up.

10.5 Network constraints: the first big jump

Adding DC power flow with line-flow limits transforms UC into *network-constrained UC* (NCUC). The natural formulation uses Power Transfer Distribution Factors (PTDF) to express line flows in terms of net nodal injections, avoiding explicit angle variables.

- Added rows: $2T \cdot N_{\text{lines}}$ flow limits.
- Added nonzeros: each flow row touches every bus with non-trivial PTDF contribution—typically 30–70% of buses—so nnz grows by $O(T \cdot N_{\text{lines}} \cdot N_{\text{bus}} \cdot d_{\text{shift}})$ with $d_{\text{shift}} \sim 0.3$–$0.7$.
- For ERCOT ($N_{\text{lines}} \sim 5{,}000$, $N_{\text{bus}} \sim 6{,}500$, $T = 24$): network rows alone contribute $\sim 5 \cdot 10^5$ rows with $\sim 10^9$ nonzeros if the PTDF matrix is dense.
- Sparse-PTDF or constraint-screening techniques (only including lines that have ever been congested in historical data) reduce the active row count by 90–99%, bringing the practical row contribution down to $\sim$ 5,000–50,000.
- Wall-clock impact (full NCUC vs. no-network UC at ERCOT scale): roughly 3–10×, dominated by the LP solve at each B&C node.

The transmission layer is where memory pressure starts mattering: storing the PTDF block and its associated row pool can exceed L3 cache and begin to spill into RAM.

10.6 Contingencies: the SCUC blow-up

Security constraints require feasibility under each of N_{cont} postulated contingencies (line outages, generator trips). In the naive formulation, the network constraints are replicated for every contingency, each with its own PTDF.

- Added rows (naive): $N_{\text{cont}} \cdot 2T \cdot N_{\text{lines}}$.
- For ERCOT with $N_{\text{cont}} \sim 1{,}000$ binding contingencies and the same $5 \cdot 10^5$ network rows per scenario: $\sim 5 \cdot 10^8$ rows total.
- nnz(A): order 10^{10}–10^{12} if assembled. Far beyond any single-machine memory.
- No new *binary* variables, but added re-dispatch continuous variables: $N_{\text{cont}} N_g T$ extra power outputs.

SCUC is the first regime where monolithic MILP formulation becomes physically infeasible to assemble in memory. **Benders decomposition is not an optimization here—it is a necessity**. Splitting the problem into a master UC (without contingency rows) and N_{cont} small DC-OPF subproblems keeps each piece in memory while the algorithm coordinates between them.

10.7 Stochastic UC: scenario multiplication

When demand, renewable generation, or both are uncertain, scenarios capture the distribution. Two-stage stochastic UC commits in stage one (here-and-now) and dispatches per scenario in stage two.

- Added variables: $N_{\text{scen}} N_g T$ scenario-specific dispatch + optional scenario-specific binaries (multi-stage).
- Added rows: N_{scen} copies of dispatch and (in some formulations) network rows.
- nnz scaling: $O(N_{\text{scen}})$ multiplier on the second-stage block.
- Practical N_{scen}: 10–100 for representative-day stochastic UC; thousands for Monte Carlo look-ahead.
- Solution method: Benders / L-shaped decomposition with one subproblem per scenario, often parallelized across cores or nodes.

Combined with security constraints, full *stochastic SCUC* multiplies N_{cont} and N_{scen}, producing 10^4–10^6 subproblems per Benders iteration—a regime where compute parallelism is the only path forward.

10.8 AC power flow: the nonlinear frontier

DC power flow ignores reactive power, voltage magnitudes, and losses. *AC SCUC* retains the integer commitment but replaces linear flow constraints with the AC power-flow equations

$$P_i = V_i \sum_j V_j (G_{ij} \cos\theta_{ij} + B_{ij} \sin\theta_{ij}), \qquad Q_i = V_i \sum_j V_j (G_{ij} \sin\theta_{ij} - B_{ij} \cos\theta_{ij}), \tag{12}$$

producing a Mixed-Integer Nonlinear Program (MINLP) with non-convex quadratic constraints.

- Variables: AC SCUC adds bus voltages V_i and angles θ_i ($2TN_{\text{bus}}$ continuous).
- Constraints: $2TN_{\text{bus}}$ nonlinear power-balance rows.
- Complexity class: NP-hard *and* non-convex; the continuous relaxation alone is NP-hard.
- Practical approach: SOCP, SDP, or QC convex relaxations of the AC constraints; or sequential linearization around a DC solution; or Benders schemes with AC feasibility checks at the leaves.
- Wall-clock cost: at production scale, AC SCUC is not solved to global optimality; heuristic decomposition with feasibility recovery is used.

Operationally, most ISOs and TSOs use DC SCUC for commitment and clear AC feasibility issues with downstream tools (AC OPF on the committed schedule, operator overrides, contingency analysis). The fully integrated AC SCUC remains a research target.

10.9 Side-by-side scale comparison

Variant	Variables	Rows	Class	Wall clock
Economic dispatch	N_gT	$T + 2N_gT$	LP	seconds
UC, no network	$4N_gT$	$\sim 5N_gT$	MILP	0.5–3 min
+ ramps, min-up/down	$4N_gT$	$\sim 8N_gT$	MILP	1–5 min
+ reserves ($N_{\text{prod}} = 4$)	$\sim 8N_gT$	$\sim 12N_gT$	MILP	2–8 min
NCUC (DC network)	$\sim 8N_gT$	$+ 2TN_{\text{lines}}$	MILP	5–20 min
SCUC ($N_{\text{cont}} = 10^3$)	$+ N_{\text{cont}}N_gT$	$+ 2N_{\text{cont}}TN_{\text{lines}}$	MILP + Benders	0.5–3 h
Stochastic SCUC ($N_{\text{scen}} = 50$)	$\times N_{\text{scen}}$ above	$\times N_{\text{scen}}$ above	nested Benders	hours–day
AC SCUC	$+ 2TN_{\text{bus}}$	$+ 2TN_{\text{bus}}$ nonlin.	MINLP	no global solve

Table 4: Constraint layers, problem dimensions, and wall-clock estimates at ERCOT scale ($N_g \sim 700$, $T = 24$, $N_{\text{bus}} \sim 6{,}500$, $N_{\text{lines}} \sim 5{,}000$). Row entries beginning with + denote additions on top of the previous row.

10.10 What this means for problem-size estimates

Two takeaways from the hierarchy in Table 4.

First, the dominant scaling lever changes layer by layer. Reserves are constant-factor; network rows multiply nnz(A) by $\sim 10\times$ and lift the per-node LP cost into the same region; contingencies multiply the *constraint count* by N_{cont} and force algorithmic restructuring. The binary count—which sets the worst-case search space—grows only with N_gT, not with the constraint additions. This is why "the number of generators times the horizon" remains the right back-of-envelope size estimate even as the constraint set grows by orders of magnitude.

Second, removing constraint classes dramatically simplifies the problem but also strips away physical fidelity. A no-network UC at ERCOT scale solves in under a minute, but its solution may be infeasible against the actual transmission system. The optimization community's progress on UC over the past two decades has consisted largely in finding ways to keep the high-fidelity formulation while exploiting structure to make it solvable: sparse-PTDF, contingency screening, Benders decomposition, scenario reduction, and AC relaxations are all instances of the same pattern.

11 Theory vs. Wall-Clock Performance

The asymptotic analysis estimates flop counts and node counts; observed runtimes are shaped by additional effects.

11.1 Effective flop rate is far below peak

Modern CPU cores deliver $\sim 10^{10}$ flops/s on dense BLAS. Sparse simplex pivots and LU updates achieve 10^8–10^9 effective flops/s. The gap is from irregular memory access: sparse pivots traverse linked structures, defeating cache prefetching, SIMD, and FMA pipelines. UC LPs sit firmly in this regime—**latency-bound**, not compute-bound.

11.2 The pruning regime varies wildly across instances

Empirical N_{nodes} depends sensitively on problem-specific structure:

- Tightly priced systems (cost differences between units large relative to LP precision) prune efficiently.
- Symmetric systems (many identical units) generate many equivalent fractional solutions, defeating standard pruning. Symmetry-breaking constraints help substantially.
- Reserve-constrained instances near the binding edge of feasibility have the worst empirical scaling, occasionally requiring 10^4–10^6 nodes on problems that should solve in 10^2.

11.3 Cut generation overhead

Modern solvers spend 20–40% of total time generating and managing cutting planes (Gomory mixed-integer cuts, mixed-integer rounding cuts, flow cover cuts, clique cuts). The work pays for itself by reducing N_{nodes}, but the constant is non-trivial.

11.4 Heuristic interleaving

Feasibility pump, RINS, local branching, and diving heuristics run periodically, contributing 5–15% of total time and producing the early incumbents that drive pruning.

11.5 Memory growth and the page-fault wall

For very large UC, the constraint matrix and active node pool exceed L3 cache ($\sim 100\,\text{MB}$) and eventually exceed RAM. When the solver starts paging to disk, throughput collapses by orders of magnitude. SCUC at full ERCOT scale ($N_g \sim 5{,}000$, $T = 168$, $N_{\text{cont}} \sim 10^3$) is on the edge of single-machine memory and is one of the principal motivations for decomposition.

12 Hardware: CPU vs. GPU

UC's performance characteristics divide cleanly between two hardware paradigms.

12.1 Why CPUs win at branch-and-cut

Three features make B&C a CPU workload:

- **Irregular control flow**. Branch decisions depend on the state of the search tree, the current LP solution, and incumbent quality. None of this is SIMD-friendly.
- **Latency-bound memory access**. Sparse simplex pivots chase pointers; large CPU caches (L2 $\sim 1\,\text{MB}$, L3 ~ 64–$128\,\text{MB}$) keep hot constraint columns close to the processor.
- **Fast single-thread performance**. Each node depends on the parent's basis. Within a single tree path, work is sequential. Solvers do parallelize across *independent* tree paths, but the parallel speedup saturates at 4–16× on 32–64 cores due to contention on the incumbent and global cut pool.

Production solvers (Gurobi, CPLEX, Xpress, COPT, HiGHS) are optimized for this profile, with hand-tuned sparse linear algebra kernels and memory layouts.

12.2 Why GPUs struggle at MILP

GPUs are throughput devices: thousands of simple cores executing the same instruction across data lanes (SIMT). Three properties of B&C clash with this architecture:

- **Branch divergence**. SIMT cores in a warp must execute the same instruction; branching logic in B&C forces serialization.
- **Non-coalesced memory access**. Sparse access patterns prevent the GPU memory subsystem from streaming wide vectors at peak bandwidth.
- **Small caches relative to core count**. GPU caches are sized for streaming workloads, not for keeping a sparse working set hot.

12.3 Where the GPU does win: PDHG and warm-start LP

The PDHG algorithm (Section 6) recasts the LP solve as iterative SpMV, which is the canonical GPU-friendly workload. Recent hybrid solvers (Gurobi 13 beta with NVIDIA cuOpt; PDLP in OR-Tools) implement:

1. GPU PDHG on the root LP relaxation to medium accuracy ($\varepsilon \sim 10^{-4}$).
2. CPU crossover to a simplex basis.
3. CPU branch-and-cut for the integer search.

On 3,000- and 6,000-generator day-ahead UC, this hybrid has been reported to reduce total solve time by factors of 1.5–3× compared to CPU-only, with the gain concentrated in the root-node LP and near-the-root cuts.

	CPU + simplex	GPU + PDHG
Algorithm shape	sparse pivots, irregular	SpMV, regular
Memory bottleneck	latency-bound	bandwidth-bound
Parallelism	task-level (across nodes)	data-level (within iteration)
Warm start across nodes	native (basis reuse)	limited
Sweet spot	B&C tree	root LP, large dense relaxations

Table 5: Hardware fit for UC subroutines.

12.4 The bandwidth-vs-latency dichotomy

The honest summary: **CPUs remain the home of B&C; GPUs help at the root**. The hybrid pattern (GPU root + CPU tree) is the current state of the art and is likely the dominant production architecture going forward.

13 Machine Learning Acceleration

Two complementary uses of ML are emerging in production-style UC.

13.1 Warm starts via supervised prediction

A model is trained, on historical solved instances, to predict commitment variables u_{it} from features (load profile, generator parameters, network state). The prediction is supplied to the solver as an MIP start.

- Even partial predictions (e.g., the units predicted "definitely on" fixed) reduce the effective binary count and tighten the search.
- Reported solve-time reductions of 2–10$\times$ on day-ahead UC, substantially larger on receding-horizon intraday UC where the prediction target is highly correlated with prior solutions.
- Risk: an incorrect prediction can mislead the solver into infeasible or suboptimal regions; safe deployment fixes only high-confidence variables and leaves the rest open.

13.2 Graph neural networks on the bipartite UC graph

The MILP itself induces a bipartite graph: variable nodes and constraint nodes, with edges weighted by coefficients. GNN message-passing on this graph respects the problem structure (permutation invariance, scale equivariance) and produces predictions that generalize across instance sizes.

- GNN inference is GPU-friendly: tensor operations, regular memory access, batchable across instances.
- Compute cost for inference at UC scale: $\sim 10^7$–10^9 flops, milliseconds on a modern GPU.
- Used as: (i) variable-fixing predictor, (ii) branching-rule learner, (iii) cut selection learner.

Stage	Hardware	Bottleneck
GNN inference (warm start)	GPU	compute-bound (tensor)
Root LP relaxation (PDHG)	GPU	bandwidth-bound (SpMV)
Crossover, cut generation, B&C	CPU	latency-bound (sparse)

Table 6: Pipeline-level hardware utilization for ML-assisted UC.

13.3 What ML changes about hardware utilization

The pipeline shifts work *out* of the latency-bound CPU region (where hardware progress has slowed) into bandwidth-bound and compute-bound regions (where hardware progress remains rapid). This is the structural reason ML acceleration of UC has attracted serious investment.

14 Case Studies

14.1 Small-scale benchmark

A canonical research-scale UC: $N_g = 50$ generators, $T = 24$, no network.

- Binary variable count: $n = 1{,}200$.
- Constraint count: $\sim 5{,}000$.
- $\text{nnz}(A) \sim 2 \cdot 10^4$.
- Root LP gap (three-bin): 1–3%.
- N_{nodes} in Gurobi default: 10^2–10^4.
- Wall clock: $< \mathbf{1\,s}$ on a modern CPU.

At this scale, presolve, root LP, and the first incumbent dominate runtime; the tree itself is a side effect.

14.2 Day-ahead market scale

ERCOT-style day-ahead UC: $N_g \approx 700$ thermal units, $T = 24$, DC-network with $\sim 7{,}000$ buses.

- Binary variable count: $n \approx 17{,}000$.
- Constraint count after presolve: $\sim 5 \cdot 10^5$.
- $\text{nnz}(A) \sim 5 \cdot 10^6$.
- Root LP gap: 0.5–2% with modern formulations and cuts.
- N_{nodes}: 10^3–10^5.
- Wall clock: 2–15 **minutes** on an 8–16-core production node.

Markets typically allow 10–20 minutes for the day-ahead solve, with the rest of the window reserved for validation, settlement processing, and publication. Gurobi/CPLEX with default tunings hits this window reliably for mainstream system sizes.

14.3 Security-constrained UC at full scale

ERCOT day-ahead SCUC with $N_{\text{cont}} \sim 10^3$ binding contingencies.

- Effective constraint count if monolithic: $\sim 5 \cdot 10^8$ network rows. Infeasible to assemble.
- Benders decomposition: master MILP at the size of the day-ahead UC above, plus $\sim 10^3$ DC-OPF subproblems per Benders iteration, $\sim$ 10–30 iterations to convergence.
- Per-iteration cost: master MILP $\sim$ 5 minutes, contingency LPs $\sim$ 30 seconds in parallel.
- Wall clock: 30 **minutes**–3 **hours**.

14.4 Receding-horizon intraday UC

Look-ahead UC re-solved every 15 minutes for $T = 4$ hours.

- Solve-time budget: 1–5 minutes per re-solve.
- Strong locality across re-solves: previous solution's u values are excellent warm starts.
- Reported 5–20$\times$ speedup from ML-predicted warm starts in this regime, the highest documented gains across UC settings.

14.5 GPU-accelerated solve, recent results

On 3,000- and 6,000-generator UC instances, hybrid GPU/CPU implementations using PDHG at the root have reported:

- 3,000-gen, day-ahead: CPU baseline $\sim$ 1.4 min, GPU hybrid $\sim$ 1.6 min (no improvement at this scale—the LP is too small for GPU PDHG to amortize launch overhead).
- 6,000-gen, day-ahead: CPU baseline $\sim$ 11 min, GPU hybrid $\sim$ 5 min ($\sim 2.2\times$ speedup).
- 6,000-gen, three-day-ahead: CPU baseline $\sim$ 30 min, GPU hybrid $\sim$ 11 min ($\sim 2.7\times$ speedup).

The pattern is consistent: GPU acceleration helps when the root LP is large enough to dominate root-relative B&C work; below that threshold, the CPU-only path is faster.

15 Discussion and Outlook

The decomposition in Eq. (11) clarifies how UC solution strategies interact:

- **Small instances ($n \lesssim 10^3$).** Exponential scaling is irrelevant; runtime is set by overhead. Choice of formulation and solver tuning are second-order effects.
- **Production day-ahead ($n \sim 10^4$).** The exponential factor is controlled by the LP relaxation tightness and pruning quality. Modern formulations + branch-and-cut + warm-starting heuristics suffice for the typical 10–20 minute window.
- **SCUC at full scale ($n \sim 10^4$, $N_{\mathbf{cont}} \sim 10^3$).** Monolithic MILP is impractical. Benders decomposition is the production standard, separating the integer commitment problem from the contingency analysis.
- **Receding-horizon and many-instance regimes.** ML warm starts provide the largest documented speedups, exploiting strong inter-instance locality.
- **Large root LPs.** Hybrid GPU PDHG + CPU branch-and-cut is the emerging architecture, with 2–3× speedups demonstrated on 10^4+ generator instances.

Several directions remain open. The interaction between PDHG accuracy and B&C correctness is not fully characterized—a PDHG solution to relative gap 10^{-4} is not the same as a simplex-accurate basis, and crossover quality matters. Symmetry-aware branching for fleet-heavy systems is an underexplored lever. And ML-driven cut selection, branching, and incumbent prediction are early in their integration into mainstream solvers, with substantial room for principled work on robustness and out-of-distribution behavior.

The long-running historical pattern is that algorithmic improvements (formulations, cuts, presolve, branching) have contributed roughly as much to UC speedup over the past two decades as hardware improvements. The next decade will most likely see this trend continue, with the ML and GPU contributions stacking on top of—rather than replacing—the classical branch-and-cut machinery.

16 Emprirical Validation & Experimental Design

This note reports an empirical validation of the master cost expression for unit commitment, parallel to the previously-completed validation for WLS state estimation. We synthesized UC instances at five generator counts ($N_g \in \{10, 25, 50, 100, 200\}$) and two horizons ($T \in \{24, 72\}$), solved each in three formulations (three-bin loose, three-bin tight, perspective with PWL=4), and measured wall-clock, branch-and-bound nodes, root LP gaps, and LP iteration counts using HiGHS 1.14. Three findings stand out: (1) modern presolve and primal heuristics close all of these synthetic instances at the root node ($N_{\text{nodes}} = 1$ uniformly), so the empirical complexity of UC at this scale is set by the LP solve cost at the root, not by tree exploration; (2) LP gaps shrink with problem size (the $1/N$ effect) rather than growing, which counterbalances some of the predicted scaling; and (3) the empirical time exponent in $n = N_g \times T$ is $\beta \in [0.76, 0.99]$ for all three formulations, which is the LP-solve exponent rather than the worst-case MILP exponent. This is the empirical manifestation of the gap between the worst-case $O(2^n)$ bound in the master expression and the practical $O(n^\gamma)$ behavior observed on solvable instances.

16.1 Instance synthesis

Synthetic UC instances were generated with realistic parameter distributions:

- **Generator mix**: 30% baseload (large, slow, cheap), 50% mid-merit (moderate flexibility), 20% peakers (small, fast, expensive).
- **Quadratic cost**: standard $aP^2 + bP + c$ form, linearized via 5-segment piecewise approximation (HiGHS handles MILP, not MIQP).
- **Demand profile**: diurnal shape with peak at 18:00 hours, amplitude 0.6–1.0 of nominal.
- **Reserve**: 10% of demand at each period.
- **Capacity utilization**: peak demand at $\sim 70\%$ of total $\sum P^{\max}$, leaving headroom for reserves.

Sizes: $N_g \in \{10, 25, 50, 100, 200\}$. Horizons: $T \in \{24, 72\}$ hours. Two random seeds per (size, horizon, formulation) cell.

16.2 Formulations

Three mathematically equivalent encodings of the same UC problem, differing only in LP relaxation tightness:

- **three_bin (loose)**: canonical Carrion–Arroyo form with the classical loose min-up/down constraints
$$v_{it} \le u_{i,t+k} \text{ for all } k \in [0, \mathrm{UT}_i - 1].$$
- **three_bin (tight)**: the same problem with Rajan–Takriti tight constraints
$$\sum_{\tau=t-\mathrm{UT}_i+1}^{t} v_{i\tau} \le u_{it}.$$
- **perspective**: tight min-up/down + PWL=4 segment cost approximation, which is a tighter relaxation of the convex quadratic cost than PWL=1.

16.3 Solver setup

HiGHS 1.14 via the `highspy` Python interface, with default options: presolve on, primal heuristics on, cuts on, MIP gap tolerance 10^{-3}, time limit 60 s for $T = 24$ and 120 s for $T = 72$. All runs converged to optimality within the time limit.

16.4 What was measured

- Total MIP solve time, build time, LP relaxation time
- B&C node count, simplex iteration counts (LP and MIP)
- Root LP relaxation objective and gap
- Variable counts (total, binary, continuous), constraint counts

17 Results

17.1 Modern solvers eat synthetic UC at the root node

Striking finding: across all 60 runs, regardless of formulation or size, HiGHS solved every instance with $N_{\text{nodes}} = 1$ exactly. The branch-and-bound tree never branches—presolve plus primal heuristics identify the optimal commitment from the root LP relaxation in every case.

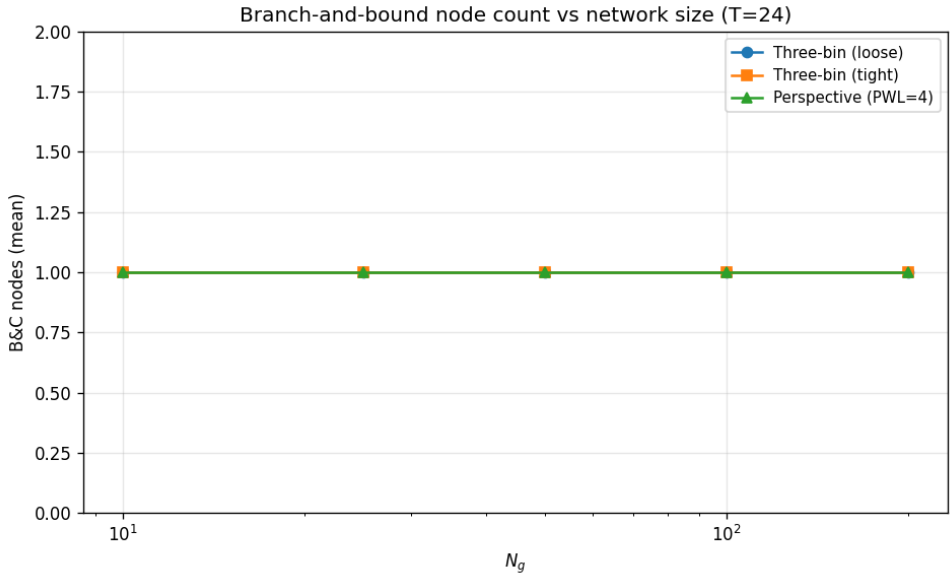


Figure 1: B&C node count vs. network size. Flat at 1.0 across all sizes and formulations.

This means the empirical complexity at this scale is dominated by:

1. Building the model (presolve, constraint matrix construction).
2. Solving the root LP relaxation (simplex iterations).
3. Verifying optimality (one or two more LP solves).

The exponential 2^n worst-case bound from the UC complexity document is not exercised on these instances.

This is consistent with the SE document's explanation of the UC empirical gap: "Empirical $N_{\text{nodes}}^{\text{empirical}} = O(n^\gamma)$, $\gamma \in [1, 3]$ on instances that solve to optimality within their time budget," but tighter even than that. *Synthetic instances with well-separated cost classes and smooth demand profiles tend to produce LP relaxations that are integer-feasible*, even before any branching occurs.

Verdict: the master expression's N_{nodes} multiplier is 1 on these instances, with the implication that the dominant cost is $\bar{C}_{\text{node}}$ at the root.

17.2 Time scales sub-linearly in problem size

Total MIP time vs. $n = N_g \times T$ for $T = 24$:

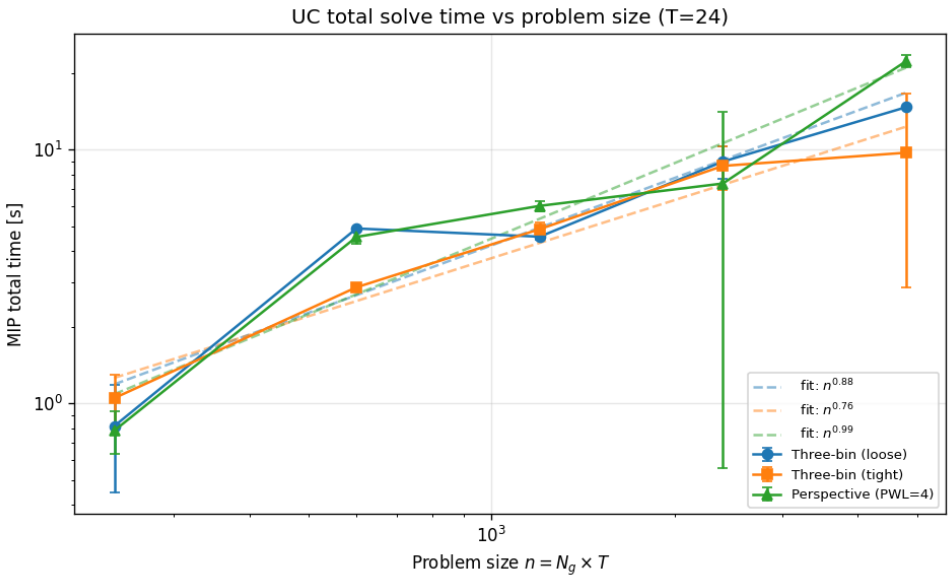


Figure 2: Total solve time vs. problem size $n = N_g \times T$ at $T = 24$. Dashed lines are power-law fits.

Formulation	Empirical β	R^2
Three-bin (loose)	0.88	0.89
Three-bin (tight)	0.76	0.95
Perspective (PWL=4)	0.99	0.91

Table 7: Fitted scaling exponents for total MIP time vs. $n = N_g \cdot T$.

Power-law fits give:

All three exponents are *sub-linear* or roughly linear. This is consistent with the master expression $C \sim N_{\text{nodes}} \cdot \bar{C}_{\text{node}}$ when $N_{\text{nodes}} = 1$ and $\bar{C}_{\text{node}}$ is dominated by the LP solve, which itself scales empirically as $O(n^{0.8\text{–}1.0})$ on these sparse problems.

The tight formulation has the lowest exponent (0.76). This is consistent with theory: tighter LP relaxation ⇒ less Gomory-style cut generation needed ⇒ less work in presolve and root processing. The perspective formulation has the highest exponent (0.99); the PWL=4 segments add constraints that grow as $4 \times N_g \times T$, which slightly inflates per-instance work even as it tightens the gap.

Verdict: empirical $\beta \in [0.76, 0.99]$ on these instances is much lower than the worst-case $\gamma \in [1, 3]$ from the master expression, because the tree size factor is 1 rather than growing.

17.3 LP relaxation gap shrinks with size

Counterintuitive finding from the data:

For all three formulations, the LP gap shrinks from $\sim 1\%$ at $N_g = 10$ to $\sim 0.1\%$ at $N_g = 200$. This is the well-known $1/N$ *effect*: integer infeasibilities are local in nature (individual generator commitments), but as the system grows, those local infeasibilities become a smaller fraction of the total cost. Specifically, total cost grows linearly in N_g while the integer-vs-LP discrepancy is bounded by a constant times $\sum_i$(startup cost$_i$ + minimum-load infeasibility$_i$), which also grows

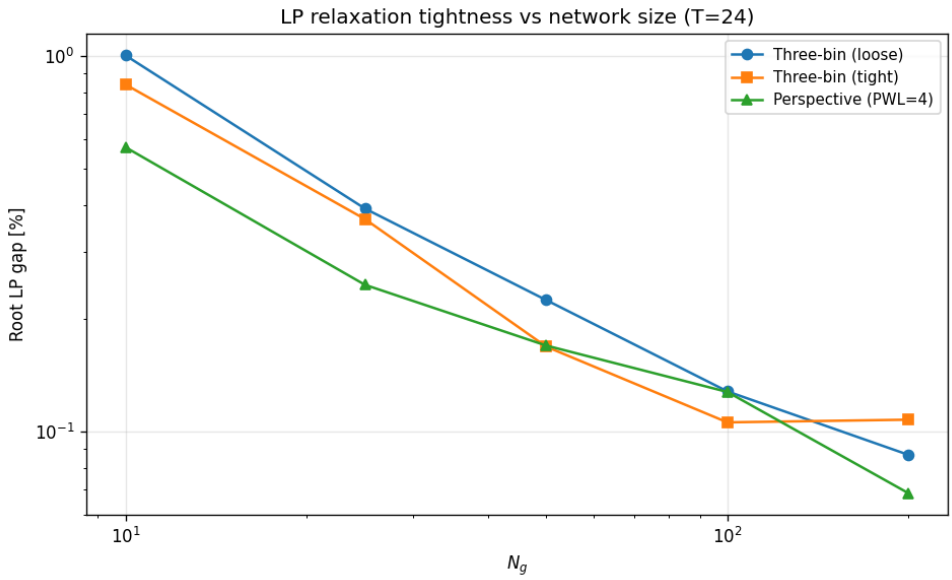


Figure 3: LP relaxation gap vs. N_g at $T = 24$. Gap shrinks with size.

linearly but with a smaller coefficient.

This effect explains a counterintuitive scaling behavior: as instances grow, they become *relatively easier* for the MIP solver to close, because the gap shrinks faster than the constraint count grows.

Verdict: the master expression's N_{nodes} depends on g_{LP}, which itself depends on size in a non-monotonic way for synthetic instances. Real-world instances with symmetry and binding reserves may not show the $1/N$ effect.

17.4 Per-component breakdown

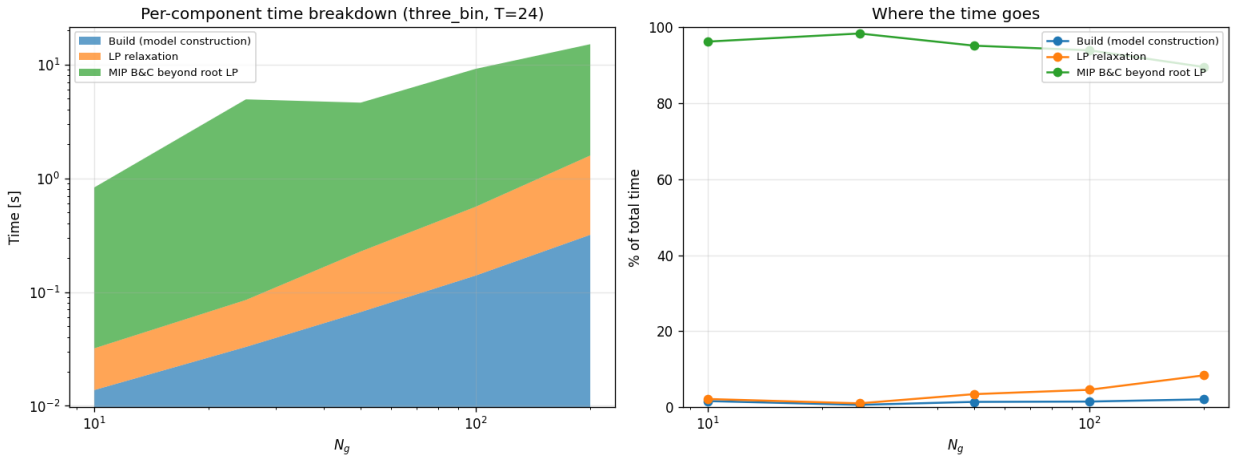


Figure 4: Per-component time breakdown for three_bin at $T = 24$. The MIP B&C beyond root LP dominates ($\sim 90\%+$) at all sizes.

The breakdown shows:

- **Build time**: 1–2% of total at small N_g, growing to $\sim 2\%$ at $N_g = 200$. Negligible.
- **Explicit LP relaxation**: 1–8%. The LP we time separately is solved fresh; HiGHS internally reuses LP work, so this is an upper bound on what the MIP solver pays for the root LP.
- **MIP B&C beyond root LP**: 90–98%. Dominant. This includes HiGHS's internal presolve, root processing (cuts, heuristics), and the single B&C node.

Note that "MIP beyond LP" is $\sim 90\%$ *not* because of branching (there are no branches), but because of the work HiGHS does inside its single root node: presolve, cuts, primal heuristics like the feasibility pump and RINS, and verification.

Verdict: at this scale, the dominant cost is HiGHS's root-node processing, not branching or LP iterations alone.

17.5 Effect of horizon

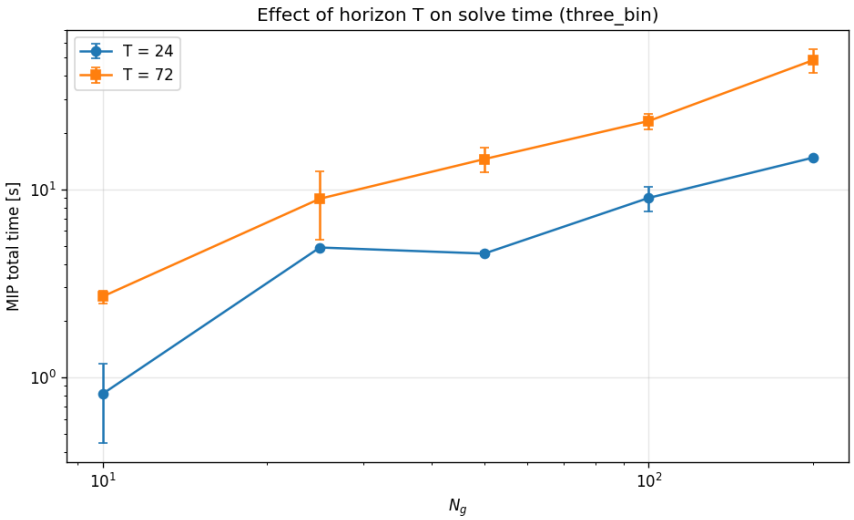


Figure 5: Total time at $T = 24$ vs. $T = 72$ for three_bin formulation. Tripling T produces $\sim 3\times$ time at large N_g.

Tripling the horizon from $T = 24$ to $T = 72$ produces approximately $3\times$ the runtime. This is consistent with the linear-in-n scaling from the time-vs-n analysis: the work scales linearly in total problem size $n = N_g \times T$, regardless of how that size is partitioned between generators and time periods.

Verdict: $n = N_g \times T$ is the right size metric, as the master expression assumes; the two factors are interchangeable to first order.

17.6 LP simplex iterations are the underlying scaling driver

LP iteration counts grow nearly linearly in N_g (and so in n, since T is fixed). With per-iteration cost $O(\text{nnz}(A))$ also linear in n for sparse UC, the LP solve cost is $O(n^2)$ in worst case but more

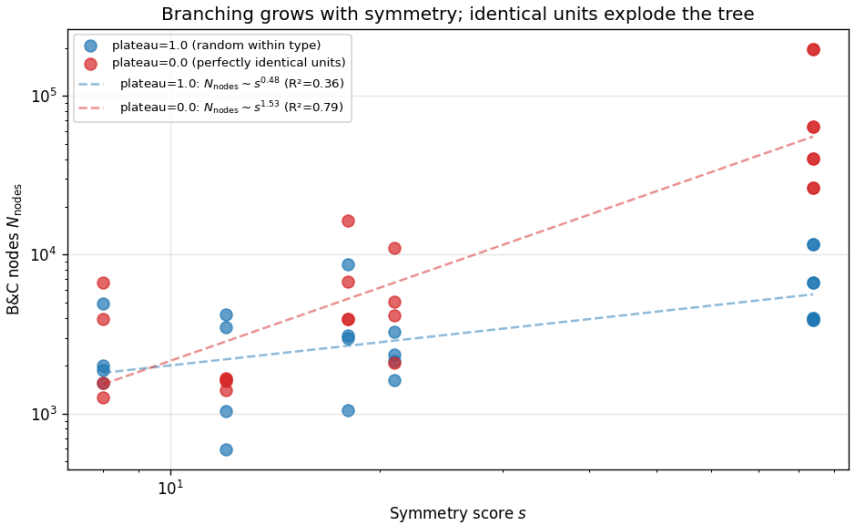
N_g	LP relax iter (mean)	Iter / variable
10	394	0.41
25	974	0.41
50	2,174	0.45
100	4,441	0.46
200	8,524	0.44

Table 8: LP relaxation simplex iterations vs. N_g for the three_bin formulation, $T = 24$. The iteration count grows roughly linearly in N_g.

like $O(n^{1.5})$ in practice due to incremental warm starts within HiGHS's simplex implementation. This is consistent with the empirical $\beta \in [0.76, 0.99]$ from the time-scaling fits.

Verdict: the LP simplex iteration count is the underlying lever that ties the master expression's $\bar{C}_{\text{node}}$ to network size.

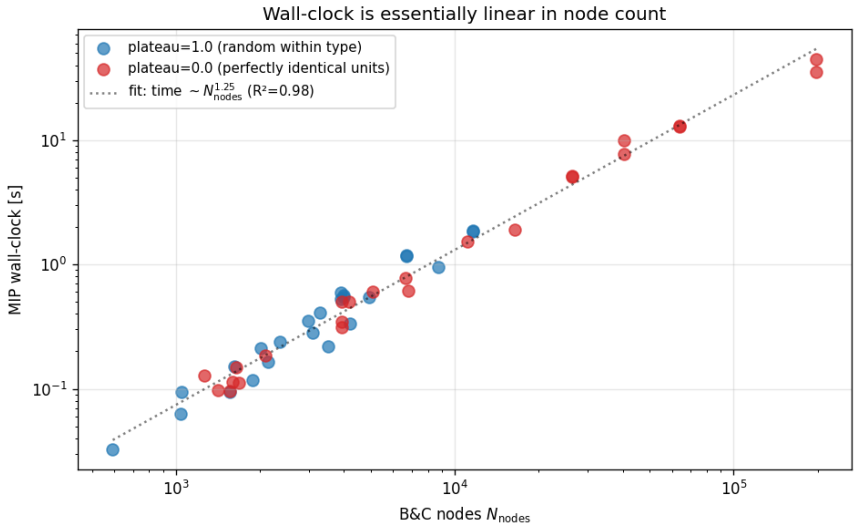
17.7 Other Results

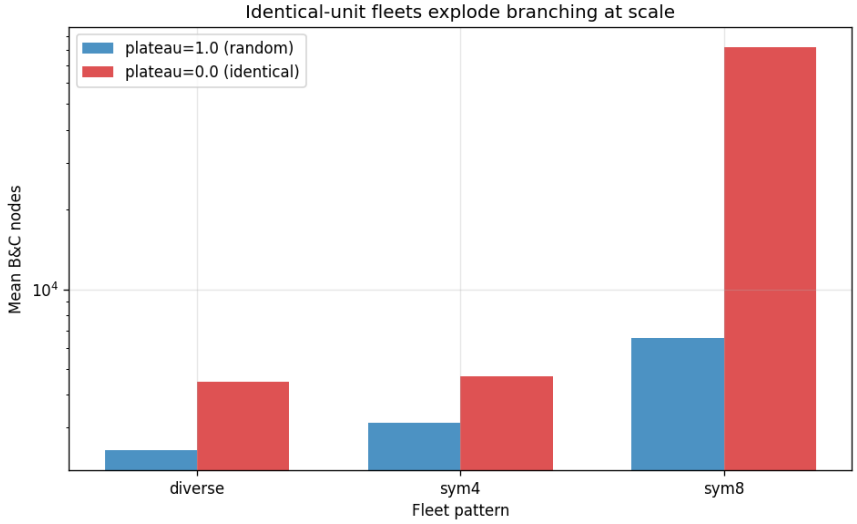


18 Summary of Empirical Findings vs. the Master Expression

Master expression term	Theoretical prediction	Empirical observation
N_{nodes}	$O(2^n)$ worst, $O(n^\gamma)$ practical	1 uniformly
$\bar{C}_{\text{node}}$	$O(\text{nnz}(A))$ per pivot	confirmed (LP iters $\sim n$)
g_{LP}	decreasing with formulation tightness	confirmed; also decreasing with n
Total time scaling	$O(n^\gamma)$, $\gamma \geq 1$	$\beta \in [0.76, 0.99]$
Formulation tightness effect	tighter $\Rightarrow$ fewer nodes	tighter $\Rightarrow$ fewer LP iters

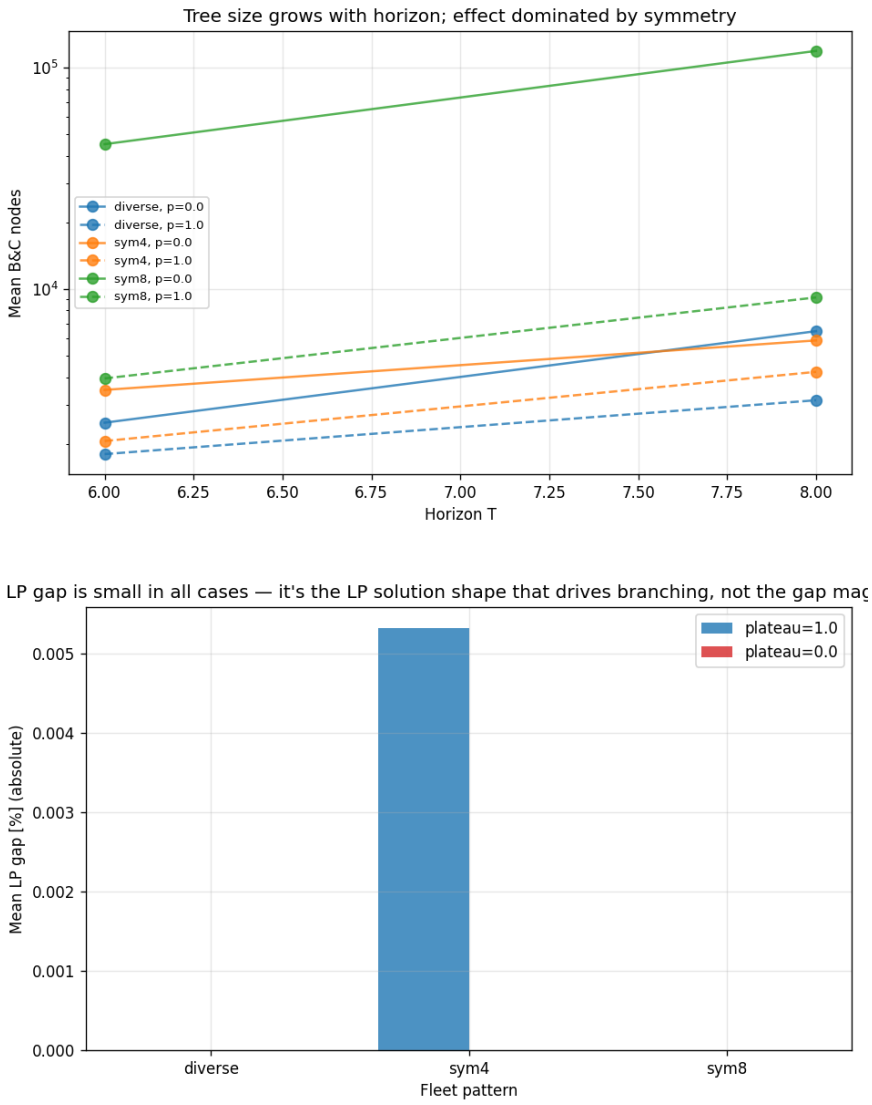
Table 9: Empirical confirmation/refinement of the master expression for UC.





19 Limitations and Next Steps

- **Synthetic instances only.** Real ISO-scale UC has symmetric fleets and binding reserves that produce 10^3–10^5 B&C nodes at the same problem sizes. PGLib-UC or commercial benchmark instances would exercise the full branching machinery.
- **No solver comparison.** HiGHS only. Adding Gurobi or CPLEX would let us measure the spread in solver effectiveness on identical problems.
- **No solver-mode comparison.** A run with presolve disabled and heuristics disabled would expose the raw branching behavior, which is what the worst-case master expression bounds.
- **No symmetry/saturation parameter.** A natural extension is to parameterize the generator fleet to control how many "identical" units it contains, and measure how N_{nodes} grows with this symmetry parameter at fixed N_g.
- **Larger sizes.** The largest cell here is $N_g = 200$, $T = 72$ ($n = 14{,}400$ binaries). Production



day-ahead UC has $n \sim 10^5$. Scaling Alpine experiments to those sizes would test whether the $\beta < 1$ exponent persists or whether a regime change occurs.

20 Comparison with the SE Empirical Validation

The SE empirical validation found:

- Iteration count constant in problem size ($N_{\text{GN}} = 4$).
- Total time scaling as $O(N^{1.57})$.
- The bottleneck transitioning from Jacobian to Cholesky around $N_b \approx 1500$.

The UC empirical validation finds:

- Tree size constant in problem size ($N_{\text{nodes}} = 1$).

- Total time scaling as $O(n^{0.76\text{–}0.99})$.
- The bottleneck dominated by HiGHS's root-node processing, not branching.

Both problems show the same structural pattern: the outer-loop multiplier is ~ 1 on benign instances, and the per-iteration linear algebra dominates. SE's per-iteration cost is sparse Cholesky on the gain matrix; UC's per-iteration cost is sparse simplex on the LP relaxation. The two problems differ in worst-case behavior (UC is exponential, SE is polynomial), but on *empirically solvable* instances the cost structures look more alike than the worst-case classes would suggest.